

Q1. Mutual Exclusion using Peterson's Algorithm

1.1 Shared Memory Setup

```
#define SHM_ADDR (MAXVA - 3*PGSIZE)
int shm_valid;
uint64 shm_pa;
```

1.2 Peterson Variables and Critical Section

```
struct shared {
    int flag[2];
    int turn;
    int shared_counter;
};
```

```
flag[i] = 1;
turn = other;
```

```
while(flag[other] == 1 && turn == other)
;
```

```
shared->shared_counter++;
```

```
flag[i] = 0;
```

- flag[i] indicates that process i wants to enter the critical section.
- turn gives the other process priority if both processes request entry.
- The while condition prevents simultaneous entry.
- The critical section increments the shared counter.
- Each process performs 10 iterations, giving an expected final value of 20.

1.3 Result

```

Sep 4 17:30
chaurya@ubuntu: ~/jvix-riscv -- make qemu CPU=1
#
#no-main -fno-builtin-printr -fno-builtin-printr -fno-builtin-printf -fno-stack-protector -fno-pie -no-pie -c -o user/peterson.o user/peterson.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/elf.ld -o user/peterson.o user/peterson.o user/objs.o user/print.o user/panic.o
riscv64-unknown-elf-objcopy -S user/peterson.o user/peterson.s
riscv64-unknown-elf-objdump -t user/peterson | sed 1,5/NOVA, TABS/d; s/ - / /; /$d/ > user/peterson.sym
#
#ls -lR /mnt: user/can user/_gch user/_forktest user/_gch user/_lnt user/_kill user/_ln user/_ls user/_phdr user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_
rphan user/_dorphan user/_sync user/_brkrstest user/peterson
meta # (boot, super, log blocks 13, inode blocks 13, bitmap blocks 1) blocks 1953 total 2986
#
#balloos: write bitmap block at sector 40
#
#user-system:riscv64-nachim virt -bios none -kernel kernel/kernel_n 128M -smp 1 -nographic -global virtio-mlnio.force:legacy=false -drive file=fs.img,if=none,format=raw,id=0 -device virtio-blk-device,drive=0,bus=virtio-mlnio-b
#
#
#xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ peterson
Process 0 in C5, counter=1
nGEMU: terminated
chaurya@ubuntu: ~/jvix-riscv -- make qemu CPU=1
#
#user-system:riscv64-nachim virt -bios none -kernel kernel/kernel_n 128M -smp 1 -nographic -global virtio-mlnio.force:legacy=false -drive file=fs.img,if=none,format=raw,id=0 -device virtio-blk-device,drive=0,bus=virtio-mlnio-b
#
#
#xv6 kernel is booting
init: starting sh
$ peterson
Process 0 in C5, counter=1
Process 0 in C5, counter=2
Process 0 in C5, counter=3
Process 0 in C5, counter=4
Process 0 in C5, counter=5
Process 0 in C5, counter=6
Process 0 in C5, counter=7
Process 0 in C5, counter=8
Process 0 in C5, counter=9
Process 0 in C5, counter=10
Process 1 in C5, counter=11
Process 1 in C5, counter=12
Process 1 in C5, counter=13
Process 1 in C5, counter=14
Process 1 in C5, counter=15
Process 1 in C5, counter=16
Process 1 in C5, counter=17
Process 1 in C5, counter=18
Process 1 in C5, counter=19
Process 1 in C5, counter=20

```

Q2.Producer–Consumer Problem (Bounded Buffer)

2.1 Semaphore Implementation

The custom xv6 sleep interface uses `sleep_prepare(channel)` followed by `sleep()`. The semaphore wait operation therefore registers the process's sleep channel before releasing the semaphore lock.

```
void
sem_wait(int id)
```

```

{
    acquire(&sems[id].lock);

    while(sems[id].value == 0){
        sleep_prepare(&sems[id]);
        release(&sems[id].lock);
        sleep();
        acquire(&sems[id].lock);
    }

    sems[id].value--;
    release(&sems[id].lock);
}

void
sem_signal(int id)
{
    acquire(&sems[id].lock);
    sems[id].value++;
    wakeup(&sems[id]);
    release(&sems[id].lock);
}

```

2.2 Producer and Consumer Logic

```

Producer:
wait(EMPTY)
wait(MUTEX)
    buffer[in] = item
    in = (in + 1) % BUFFER_SIZE
signal(MUTEX)
signal(FULL)

```

```

Consumer:
wait(FULL)
wait(MUTEX)
    item = buffer[out]
    out = (out + 1) % BUFFER_SIZE
signal(MUTEX)
signal(EMPTY)

```

The producer generates items 1–20 and the consumer removes 20 items. The shared buffer is obtained with `shm_get()` before `fork()`, and the existing `kfork()` shared-memory mapping allows both processes to access the same page.

2.3 Results

```

shaurya@shaurya6480:~/xv6-peterson$ make qemu
Producer: 5
Consumer: 4
Producer: 6
Consumer: 5
Producer: 7
Consumer: 6
Producer: 8
Consumer: 7
Producer: 9
Consumer: 8
Producer: 10
Consumer: 9
Producer: 11
Consumer: 10
Producer: 12
Consumer: 11
Producer: 13
Consumer: 12
Producer: 14
Consumer: 13
Producer: 15
Consumer: 14
Producer: 16
Consumer: 15
Producer: 17
Consumer: 16
Producer: 18
Consumer: 17
Producer: 19
Consumer: 18
Producer: 20
Consumer: 19
Consumer: 20
Producer-Consumer completed

```

Q3.Readers–Writers Problem

3.1 Shared Structure

```
struct shared_rw {  
    int shared_data;  
    int read_count;  
};
```

3.2 Reader Logic

```
wait(TURNSTILE)  
signal(TURNSTILE)
```

```
wait(R_MUTEX)  
read_count++
```

```
if(read_count == 1)  
    wait(RESOURCE)
```

```
signal(R_MUTEX)
```

```
READ shared_data
```

```
wait(R_MUTEX)  
read_count--
```

```
if(read_count == 0)  
    signal(RESOURCE)
```

```
signal(R_MUTEX)
```

The first reader locks RESOURCE and subsequent readers can enter concurrently. The last reader releases RESOURCE. R_MUTEX protects changes to read_count.

3.3 Writer Logic

```
wait(TURNSTILE)  
wait(RESOURCE)
```

```
shared_data++
```

```
signal(RESOURCE)  
signal(TURNSTILE)
```

RESOURCE is held for the complete write operation, so no other reader or writer can enter the resource section during a write.

3.4 Reader Priority and Writer Starvation

Pure reader-preference Readers–Writers can starve writers if readers continuously arrive. The TURNSTILE mechanism was added to satisfy the requirement that no writer should starve indefinitely: once a writer obtains the turnstile, new readers cannot continuously pass it while the writer waits.

3.5 Process Configuration

- 3 reader processes and 2 writer processes are created using fork().
- Each reader performs 3 reads.
- Each writer performs 3 increments.
- Therefore the final shared_data value is expected to be $2 \times 3 = 6$.
- PID and uptime timestamps are printed for entry, exit, and write events.

3.6 Result

```
shaurya@Shaurya6480:~/xv6-peterson$ make qemu CPUS=2
init: starting sh
$ readwrite
[Reader] PID=4 Time=49 ENTER shared_data=0 ActiveReaders=1
[Reader] PID=5 Time=50 ENTER shared_data=0 ActiveReaders=2
[Reader] PID=4 Time=51 EXIT ActiveReaders=1
[Reader] PID=5 Time=51 EXIT ActiveReaders=0
[Reader] PID=6 Time=52 ENTER shared_data=0 ActiveReaders=1
[Reader] PID=6 Time=52 EXIT ActiveReaders=0
[Writer] PID=7 Time=53 ENTER shared_data=0
[Writer] PID=7 Time=53 WRITE shared_data=1
[Writer] PID=7 Time=54 EXIT
[Reader] PID=4 Time=54 ENTER shared_data=1 ActiveReaders=1
[Reader] PID=5 Time=55 ENTER shared_data=1 ActiveReaders=2
[Reader] PID=4 Time=55 EXIT ActiveReaders=1
[Reader] PID=5 Time=56 EXIT ActiveReaders=0
[Writer] PID=7 Time=56 ENTER shared_data=1
[Writer] PID=7 Time=57 WRITE shared_data=2
[[WReradietr] erP] IDP=6I DTim=7e T=i5m7e =E57N ETXEIRT
shared_data=2 ActiveReaders=1
[Reader] PID=6 Time=58 EXIT ActiveReaders=0
[Writer] PID=7 Time=59 ENTER shared_data=2
[Writer] PID=7 Time=59 WRITE shared_data=3
[Writer] PID=7 Time=60 EXIT
[Reader] PID=4 Time=60 ENTER shared_data=3 ActiveReaders=1
[Reader] PID=5 Time=61 ENTER shared_data=3 ActiveReaders=2
[Reader] PID=4 Time=61 EXIT ActiveReaders=1
[Reader] PID=5 Time=62 EXIT ActiveReaders=0
[Reader] PID=6 Time=62 ENTER shared_data=3 ActiveReaders=1
[Reader] PID=6 Time=63 EXIT ActiveReaders=0
[Writer] PID=8 Time=63 ENTER shared_data=3
[Writer] PID=8 Time=64 WRITE shared_data=4
[Writer] PID=8 Time=64 EXIT
[Writer] PID=8 Time=65 ENTER shared_data=4
[Writer] PID=8 Time=65 WRITE shared_data=5
[Writer] PID=8 Time=66 EXIT
[Writer] PID=8 Time=66 ENTER shared_data=5
[Writer] PID=8 Time=67 WRITE shared_data=6
[Writer] PID=8 Time=67 EXIT

Final shared data = 6
Readers-Writers completed
```

4. Conclusion

Peterson's Algorithm was implemented to provide mutual exclusion between two processes using shared memory. A shared-memory mechanism using `shm_get()` was added so the parent and child could access the same variables. The shared memory contains `flag[2]`, `turn`, and `shared_counter`. Each process sets its flag, gives the other process the turn, and waits until it is safe to enter the critical section. The counter is incremented inside the critical section and the flag is cleared afterward. With 10 iterations per process, the final counter reached **20**, showing that no updates were lost and mutual exclusion was achieved.

A bounded circular buffer of size 5 was implemented using shared memory. Three semaphores were used: `EMPTY` = 5, `FULL` = 0, and `MUTEX` = 1. The producer waits for an empty slot and exclusive buffer access before inserting items, while the consumer waits for an available item and exclusive access before removing it. The semaphore implementation uses `xv6`'s `sleep_prepare()`, `sleep()`, and `wakeup()` mechanisms. The producer generated items **1–20**, and the consumer successfully received them in order without loss or duplication.

The Readers–Writers problem was implemented using shared memory, `read_count`, and three semaphores: `R_MUTEX`, `RESOURCE`, and `TURNSTILE`. `R_MUTEX` protects `read_count`, while `RESOURCE` provides exclusive access for writers. Multiple readers can access `shared_data` simultaneously, with the first reader locking the resource and the last reader releasing it. `TURNSTILE` prevents writers from being indefinitely starved by continuously arriving readers. Three readers and two writers were created using `fork()`. The output showed multiple active readers and exclusive writer access, with the final `shared_data` value reaching **6**.